PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

1. Паспорт занятия

Содержание

- 2. Образовательные результаты
- 4. Исходный комплект проекта
- 5. Требуемая структура рабочего продукта
- 6. Исходная логика продукта
- 7. План занятия
- 9. Что необходимо сдать
- 10. Критерии оценивания
- 11. Итоговая рефлексия

Работа над программным продуктом не заканчивается после написания последней функции. Если файлы названы непонятно, версии не согласованы, а условия запуска известны только автору, даже исправный код трудно проверить и передать. На занятии предстоит превратить набор разрозненных материалов в комплект, пригодный для демонстрации и повторного использования.

Параметр	Содержание
Цель	Подготовить рабочий продукт и документацию к воспроизводимой демонстрации и передаче другому пользователю.
Итог	Структурированная папка проекта, проверенный код или шаблон, паспорт датасета, отчёт о тестировании, README и инструкция по запуску.
Исходный проект	ServiceDesk Assistant 2.0 — учебный классификатор заявок технической поддержки.
Продолжительность	90 минут.
Форма работы	Индивидуально или в парах по решению преподавателя.

Параметр	Содержание
Главный критерий	Другой человек может понять назначение проекта, запустить продукт и воспроизвести демонстрационный результат.

Рабочий продукт — это не один файл
Готовность проекта определяется не только тем, запускается ли код у автора. Необходимы исходные данные, зафиксированная версия, результаты тестов, инструкция, ограничения и понятная структура папки. Такой комплект позволяет показать продукт, проверить его и продолжить работу без устных пояснений автора.

> **Важно**
>
> Воспроизводимость не означает, что продукт всегда выдаёт желаемый результат. Она означает, что другой пользователь при тех же версиях кода и данных может повторить описанные шаги, увидеть результат и сопоставить его с отчётом.

2. Образовательные результаты

Результаты занятия связаны между собой: структура папок помогает найти нужные материалы, версии подтверждают их согласованность, тестовый отчёт показывает качество, а README объясняет порядок проверки.

- соберёте единый комплект рабочего продукта;
- проверите соответствие кода, датасета и документации одной версии;
- подготовите паспорт данных и результатов тестирования;
- напишете воспроизводимую инструкцию по запуску или использованию;
- исправите ошибку в коде подготовки итогового отчёта;
- зафиксируете ограничения, известные проблемы и дальнейшие шаги.

К завершению работы каждый заявленный результат должен подтверждаться конкретным артефактом: файлом, заполненной таблицей, исправленным фрагментом кода или отметкой в чек-листе.

4. Исходный комплект проекта

Команда получила набор файлов после разработки и тестирования. В нём есть полезные материалы, но структура и названия не позволяют сразу понять, какая версия является рабочей.
Перед переименованием необходимо определить роль каждого файла: относится ли он к рабочему коду, данным, результатам, документации или архиву. Переименование без такого аудита может скрыть проблему, но не устранить её.

Текущее имя	Содержание	Проблема
`main_final2.py`	Код классификатора	Название не содержит продукта и версии.
`tickets_new.csv`	Датасет из 64 заявок	Не указаны назначение и версия.
`results.csv`	Результаты тестирования	Неясно, для какой версии кода получены результаты.
`notes.docx`	Черновые выводы	Нет структуры и критериев качества.
`run.txt`	Несколько команд запуска	Нет зависимостей, ожидаемого результата и решения ошибок.
`old_model.py`	Предыдущая версия	Может быть случайно запущена вместо рабочей.

> **Частая ошибка**
>
> Слова `final`, `new` и `latest` не заменяют номер версии. Через некоторое время по таким названиям невозможно надёжно определить порядок файлов. Для рабочего комплекта лучше использовать устойчивую схему именования с назначением и версией.

5. Требуемая структура рабочего продукта

Структура папок разделяет материалы по назначению. Благодаря этому рабочий код не смешивается со старыми версиями, исходные данные — с результатами обработки, а инструкции — с временными заметками.

Элемент	Назначение	Минимальное содержание
`src/`	Рабочий код	Основной файл, версия, точка запуска.
`data/`	Данные	Рабочий датасет, версия и паспорт данных.
`tests/`	Проверки	Набор тестов или таблица сценариев.
`results/`	Результаты	Метрики, журнал ошибок, дата и версия.
`docs/`	Документация	README, инструкция, ограничения, выводы.
`archive/`	Архив	Предыдущие версии, не участвующие в показе.

Папка `archive/` нужна не для удаления истории, а для отделения истории от текущего состояния проекта. Файлы из неё сохраняются для справки, но не должны использоваться в демонстрационном запуске.

Правило согласованности версии
Код, датасет, тесты и отчёт должны ссылаться на одну версию продукта. Например: ServiceDesk Assistant 2.0, dataset v1.3, tests v2.0. Если отчёт получен на другом датасете или коде, его нельзя выдавать за результат текущей версии.

> **Обрати внимание**
>
> У компонентов могут быть разные номера версий: продукт 2.0 и датасет 1.3 не обязаны называться одинаково. Согласованность означает, что в документации точно указано, какая версия каждого компонента участвовала в конкретном запуске и тестировании.

6. Исходная логика продукта

Классификатор получает текст заявки и значение уверенности. Сначала он проверяет входной текст, затем нормализует его и ищет ключевые слова четырёх категорий. Автоматическая маршрутизация выполняется только тогда, когда найдена ровно одна категория и уверенность не ниже установленного порога.

```
def classify_request(text, confidence):
if not isinstance(text, str) or not text.strip():
return {"category": "other", "action": "input_error"}
normalized = text.strip().lower()
categories = {
"access": ["пароль", "вход", "доступ", "учётная запись"],
"network": ["сеть", "wi-fi", "интернет", "сервер недоступен"],
"equipment": ["принтер", "ноутбук", "компьютер", "клавиатура"],
"software": ["программа", "приложение", "обновление", "ошибка установки"],
}
matched = [
category
for category, words in categories.items()
if any(word in normalized for word in words)
]
if len(matched) != 1:
return {"category": "other", "action": "operator_review"}
action = "operator_review" if confidence < 0.60 else "auto_route"
return {"category": matched[0], "action": action}
```

Если текст пустой или состоит только из пробелов, функция возвращает действие `input_error`. Если не найдено ни одной категории либо одновременно найдены несколько категорий, заявка направляется оператору с категорией `other`.

> **Граничное условие**
>
> Проверка использует строгое сравнение `confidence < 0.60`. Поэтому значение `0.60` уже допускает `auto_route`, если найдена ровно одна категория. Значение ниже `0.60` приводит к `operator_review`.

Перед тестированием полезно сформулировать ожидаемый результат без запуска программы. Например, фраза с ключевыми словами из двух разных категорий должна перейти к оператору, поскольку условие `len(matched) != 1` не выполнено для единственной категории.

7. План занятия

Этапы выстроены в порядке зависимостей: сначала определяется состав актуальной версии, затем проверяются данные и код, после этого оформляются результаты и инструкция.

Приёмочная проверка выполняется последней, поскольку она оценивает уже собранный комплект.

Этап	Содержание	Время
Старт	Вопросы и аудит исходного комплекта.	10 мин
Задание 1	Структура папки, имена и версии.	15 мин
Задание 2	Паспорт датасета и код продукта.	20 мин
Задание 3	Отчёт о тестировании и анализ вывода.	20 мин
Задание 4	README и инструкция по запуску.	15 мин
Задание 5	Приёмочная проверка комплекта.	10 мин

9. Что необходимо сдать

Все элементы сдачи должны относиться к согласованной версии комплекта. Если документ подготовлен для другой версии, это необходимо явно обозначить и не включать его в текущий демонстрационный результат.

☐ структуру папок и новые имена файлов;
☐ журнал версий;
☐ паспорт датасета;
☐ демонстрационную таблицу тестов;
☐ прогноз вывода кода;
☐ исправленную функцию `make_summary`;
☐ README и пошаговую инструкцию;
☐ заполненный чек-лист приёмки.

Перед сдачей следует открыть комплект как новый пользователь: начать с README, найти точку входа, определить версии и проверить, что указанные в документации пути действительно существуют.

10. Критерии оценивания

Оценивание учитывает не только наличие файлов, но и возможность проверить их содержание. Пустой шаблон, неподтверждённая метрика или инструкция без ожидаемого результата не считаются полным выполнением соответствующего критерия.

Критерий	Баллы	Полное выполнение
Структура и версии	3	Файлы распределены, имена понятны, версии согласованы.
Паспорт датасета	4	Описаны поля, происхождение, проверки и ограничения.
Отчёт и тесты	4	Есть версия, ожидание, факт, метрики и вывод.
Код и исправление ошибки	3	Прогноз верен, `make_summary` корректна и обрабатывает неверный ввод.
README и инструкция	4	Другой пользователь может запустить и проверить продукт.
Приёмочная проверка	2	Чек-лист заполнен, выявленные проблемы зафиксированы.

11. Итоговая рефлексия

Ответы должны опираться на подготовленный комплект, а не только на общие определения. Полезно указывать конкретный файл, правило версий, ограничение или шаг инструкции.

Вопрос	Ответ
Какой файл является точкой входа в комплект?	
Что делает результат воспроизводимым?	
Какое ограничение продукта наиболее важно сообщить пользователю?	
Какой шаг документации Вы бы автоматизировали?	

> **Мини-проверка**
>
> Можно ли считать комплект готовым, если код запускается у автора, но версия датасета в отчёте не указана? Обоснуйте ответ через главный критерий занятия и правило согласованности версий.